Kraken (Kraken Pro) — security stack assessment (2026-10-31)

Subject: the security/anti-bot mechanisms on `kraken.com` and `pro.kraken.com/app`
Method: public-site analysis + runtime observation of the unauthenticated app. All 3rd-party telemetry observed or locally fulfilled; no account, no auth, nothing forwarded.
Result: Kraken runs **Cloudflare at the edge + a real client-side proof-of-work (PoW) challenge** on sensitive requests (`/pow` → double SHA-256 leading-zero nonce → `x-pow` header), WebAuthn/passkey step-up, a first-party risk/intel API (`iapi.kraken.com`), and the standard analytics stack. The interesting find: **the one "client-computed" defense Kraken ships is proof-of-work — a legitimately hard-to-fake cost mechanism, not a fingerprint.** The app client itself is clean (no device fingerprint bundle).

---

1. The stack

Layer	Product	Evidence
Edge	**Cloudflare**	`server: cloudflare`, `cf-ray`, `cf-cache-status` on www + pro
Anti-bot (on-demand)	**Kraken in-house PoW**	`/pow` endpoint → double-SHA-256 nonce search → `x-pow` header (in `frontend.js`)
Auth step-up	**WebAuthn / passkey**	`/devices/bound-keys/challenge`, passkey flows
Risk/intel API	Kraken first-party	`iapi.kraken.com/api/internal/*` (session, user-context, markets, kyc)
Analytics	Segment, GA, Braze, Twitter/LinkedIn/Reddit/Snap pixels, OneTrust consent	`cdn.seg.kraken.com`, `analytics.*`, `*ads*` hosts
Trading UI	clean React app (rspack)	`pro.kraken.com/app`, no client fingerprint bundle

2. The proof-of-work challenge (the interesting defense)

Recovered from `frontend.js` — this is the real "client-computed but good" pattern:

```
// 1) server issues the challenge (GET /pow → { data, randomness, difficulty })
// 2) client solves: find nonce whose double-SHA-256 has `difficulty` leading zero bits
m = async data => crypto.subtle.digest("SHA-256", await crypto.subtle.digest("SHA-256", data)); // DOUBLE
...
let o = 1;
do {
  crypto.getRandomValues(n);                    // random nonce
  s.set(n, i.length);                           // challenge.data || nonce
  r = new DataView(await m(s));
  o++;
} while (r.getUint32(0) >> (32 - e.difficulty) != 0);   // leading-zero-bits check
// 3) solution hex sent as request header
headers: { "x-pow": solution }
```

- **What it proves:** the client burned real CPU (expected `2^difficulty` hash attempts). It's a **cost mechanism**, not a secret or a fingerprint.
- **When it fires:** sensitive requests (e.g., logout; server-issued on-demand when risk wants proof).
- **Why it's the "good" client-computed defense:** unlike a fingerprint hash (which can be replayed/tampered), PoW is **fresh per challenge** (server supplies `randomness`) and **verifiable server-side** (re-hash the solution). The client's answer is worthless without doing the work. Replay is blocked by the fresh challenge; tampering is caught by the check.

3. WebAuthn / passkey step-up

Alongside PoW, Kraken uses `/devices/bound-keys/challenge` + passkey (WebAuthn) flows for sensitive actions — a **hardware-backed** proof that a client fingerprint can't substitute for. This is the strongest auth tier in the stack.

4. Risk/intel (server-side)

`iapi.kraken.com/api/internal/*` is the first-party risk/intel layer — session state, user-context features, markets, KYC attributes. The decision logic is **server-side** (the client just reads the responses). No client-mintable risk token observed.

5. The tracking surface (unauthenticated app)

Standard commercial stack: Segment, Google Analytics, Braze (device_id), Twitter/LinkedIn/Reddit/Snapchat/Google-Ads pixels, OneTrust consent + geolocation, Upsellit, DoubleClick floodlights. Same surveillance web as publishers, nothing unusual.

6. How it compares (the learning)

Defense	Type	Verdict
Cloudflare edge	transport/JS gate	Good (buy the hard part)
PoW (`x-pow`)	**client cost proof**	**Good** — fresh per request, server-verified, can't be replayed/tampered
WebAuthn/passkey	hardware proof	Good — strongest auth tier

Defense	Type	Verdict
Risk API (`iapi`)	server-side scoring	Good — decision stays server-side
Analytics pixels	tracking	Neutral (commercial standard)
Client fingerprint	none shipped	**Notable** — Kraken doesn't rely on a client device hash

The honest take: Kraken's client has **no fingerprint bundle** — unlike Binance (client-computed `BNC_FV_KEY`) or the device-intel SDKs. Its one client-computed defense is **proof-of-work**, which is the *right* kind: server-issued, fresh, verifiable, cost-based. That's the cleanest example in this whole project of "client does work, server verifies" done correctly.

7. Honest boundaries

- PoW solver recovered statically; the `/pow` challenge itself only fires on auth-sensitive requests (we're unauthenticated), so we did **not** capture a live solve or the `x-pow` on the wire — but the client code is complete and unambiguous.
- No account/auth touched; everything observed or locally fulfilled.

8. Artifacts

File	What
`assets/frontend.js`	the 3.9 MB app frontend (PoW solver + challenge paths)
`assets/main.js` , `assets/vendor.js`	app bundles
`scripts/kr-probe.mjs` , `kr-probe2.mjs`	www + app probes
`artifacts/probe1.json` , `probe2.json`	request/host maps
`artifacts/pow-reqs.json`	PoW-related request capture

9. Summary line

> *"Kraken = Cloudflare edge + an on-demand client-side proof-of-work challenge (`/pow` → double-SHA-256 leading-zero nonce → `x-pow` header) + WebAuthn passkey step-up + a server-side risk API. No client fingerprint bundle. The PoW is the rare 'client-computed' defense that's actually good: fresh per request, server-verified, cost-based — the client's answer is worthless without doing the work."*